```java
package com.codes.java;

import com.codes.java.codingquestions.arrays.MajorityElement;

import java.util.*;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

public class ArrayCodingQuestionsCode {


    // ============================================================================
    =
    // Question: Find Largest Element
    // Input : {10, 20, 30, 40, 50}
    // Output: 50
    // Time Complexity : O(n)
    // Space Complexity: O(1)

    // ============================================================================
    =
    public static int findLargest(int[] arr) {

        if (arr == null || arr.length == 0)
            throw new IllegalArgumentException("Array is empty");

        int largest = arr[0];

        for (int num : arr) {
            if (num > largest) {
                largest = num;
            }
        }

        return largest;
    }


    // ============================================================================
    =
    // Question: Find Smallest Element
    // Input : {10, 20, 30, 40, 50}
    // Output: 10
    // Time Complexity : O(n)
    // Space Complexity: O(1)

    // ============================================================================
    =
    public static int findSmallest(int[] arr) {

        if (arr == null || arr.length == 0)
            throw new IllegalArgumentException("Array is empty");

        int smallest = arr[0];

```

```
50          for (int num : arr) {
51              if (num < smallest) {
52                  smallest = num;
53              }
54          }
55
56          return smallest;
57      }
58
59
    // ================================================================
    ==
60      // Question: Find Second Largest Element
61      // Input : {10, 20, 30, 40, 50}
62      // Output: 40
63      // Time Complexity : O(n)
64      // Space Complexity: O(1)
65
    // ================================================================
    ==
66      public static int findSecondLargest(int[] arr) {
67
68          if (arr == null || arr.length < 2)
69              throw new IllegalArgumentException("Array must have at least 2
    elements");
70
71          int largest       = Integer.MIN_VALUE;
72          int secondLargest = Integer.MIN_VALUE;
73
74          for (int num : arr) {
75              if (num > largest) {
76                  secondLargest = largest;
77                  largest = num;
78              } else if (num > secondLargest && num != largest) {
79                  secondLargest = num;
80              }
81          }
82
83          if (secondLargest == Integer.MIN_VALUE)
84              throw new IllegalArgumentException("No second largest element
    exists");
85
86          return secondLargest;
87      }
88
89
    // ================================================================
    ==
90      // Question: Find Second Smallest Element
91      // Input : {10, 20, 30, 40, 50}
92      // Output: 20
93      // Time Complexity : O(n)
94      // Space Complexity: O(1)
95
    // ================================================================
    ==
96      public static int findSecondSmallest(int[] arr) {
```

```java

        if (arr == null || arr.length < 2)
            throw new IllegalArgumentException("Array must have at least 2 elements");

        int smallest       = Integer.MAX_VALUE;
        int secondSmallest = Integer.MAX_VALUE;

        for (int num : arr) {
            if (num < smallest) {
                secondSmallest = smallest;
                smallest = num;
            } else if (num < secondSmallest && num != smallest) {
                secondSmallest = num;
            }
        }

        if (secondSmallest == Integer.MAX_VALUE)
            throw new IllegalArgumentException("No second smallest element exists");

        return secondSmallest;
    }


    // ============================================================================
    // Question: Reverse Array
    // Input : {1, 2, 3, 4, 5}
    // Output: {5, 4, 3, 2, 1}
    // Time Complexity : O(n)
    // Space Complexity: O(1)

    // ============================================================================
    public static void reverseArray(int[] arr) {

        if (arr == null || arr.length <= 1) return;

        int left  = 0;
        int right = arr.length - 1;

        while (left < right) {
            int temp   = arr[left];
            arr[left]  = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }


    // ============================================================================
    // Question: Sum Of Array Elements
    // Input : {1, 2, 3, 4, 5}
    // Output: 15
```

```
146     // Time Complexity : O(n)
147     // Space Complexity: O(1)
148 
    // ================================================================
    ==
149     public static int sumOfElements(int[] arr) {
150 
151         if (arr == null) return 0;
152 
153         int sum = 0;
154         for (int num : arr) {
155             sum += num;
156         }
157 
158         return sum;
159     }
160 
161 
    // ================================================================
    ==
162     // Question: Average Of Array Elements
163     // Input : {1, 2, 3, 4, 5}
164     // Output: 3.0
165     // Time Complexity : O(n)
166     // Space Complexity: O(1)
167 
    // ================================================================
    ==
168     public static double averageOfElements(int[] arr) {
169 
170         if (arr == null || arr.length == 0) return 0.0;
171 
172         return (double) sumOfElements(arr) / arr.length;
173     }
174 
175 
    // ================================================================
    ==
176     // Question: Count Even And Odd Numbers
177     // Input : {1, 2, 3, 4, 5, 6}
178     // Output: Even = 3, Odd = 3
179     // Time Complexity : O(n)
180 
    // ================================================================
    ==
181     public static void countEvenOdd(int[] arr) {
182 
183         int even = 0;
184         int odd  = 0;
185 
186         if (arr == null) {
187             System.out.println("    Even Count : " + even);
188             System.out.println("    Odd Count  : " + odd);
189             return;
190         }
191 
192         for (int num : arr) {
```

```
193             if (num % 2 == 0) {
194                 even++;
195             } else {
196                 odd++;
197             }
198         }
199 
200 //         long even = Arrays.stream(arr)
201 //                 .filter(num -> num % 2 == 0)
202 //                 .count();
203 //
204 //         long odd = Arrays.stream(arr)
205 //                 .filter(num -> num % 2 != 0)
206 //                 .count();
207 
208 //         Map<Boolean, Long> counts = Arrays.stream(arr)
209 //                 .boxed()
210 //                 .collect(Collectors.partitioningBy(
211 //                         x -> x % 2 == 0,
212 //                         Collectors.counting()
213 //                 ));
214 //
215 //         long even = counts.get(true);
216 //         long odd = counts.get(false);
217 
218         System.out.println("    Even Count : " + even);
219         System.out.println("    Odd Count  : " + odd);
220     }
221 
222 
    // ================================================================
    ==
223     // Question: Find Maximum And Minimum
224     // Input : {10, 20, 30, 40, 50}
225     // Output: Max = 50, Min = 10
226     // Time Complexity : O(n)
227     // Space Complexity: O(1)
228 
    // ================================================================
    ==
    --
229     public static int[] findMaxAndMin(int[] arr) {
230 
231         if (arr == null || arr.length == 0)
232             throw new IllegalArgumentException("Array is empty");
233 
234         int max = arr[0];
235         int min = arr[0];
236 
237         for (int num : arr) {
238             if (num > max)
239                 max = num;
240             if (num < min)
241                 min = num;
242         }
243 
244         return new int[]{max, min};
245     }
```

```
246
247
    // ================================================================
    ==
248     // Question: Search Element In Array
249     // Input : {10, 20, 30, 40, 50}, target = 30
250     // Output: true
251     // Time Complexity : O(n)
252     // Space Complexity: O(1)
253
    // ================================================================
    ==
254     public static boolean searchElement(int[] arr, int target) {
255
256         if (arr == null)
257             return false;
258
259         for (int num : arr) {
260             if (num == target)
261                 return true;
262         }
263
264         return false;
265     }
266
267
    // ================================================================
    ==
268     // Question: Remove Duplicates From Array
269     // Input : {1, 2, 2, 3, 4, 4, 5}
270     // Output: [1, 2, 3, 4, 5]
271     // Time Complexity : O(n)
272     // Space Complexity: O(n)
273
    // ================================================================
    ==
274     public static int[] removeDuplicates(int[] arr) {
275
276         if (arr == null) return new int[0];
277
278         return Arrays.stream(arr).distinct().toArray();
279     }
280
281
    // ================================================================
    ==
282     // Question: Find Duplicate Elements
283     // Input : {1, 2, 2, 3, 4, 4, 5}
284     // Output: [2, 4]
285     // Time Complexity : O(n)
286     // Space Complexity: O(n)
287
    // ================================================================
    ==
288     public static Set<Integer> findDuplicates(int[] arr) {
289
290         Set<Integer> set       = new HashSet<>();
```

```
291         Set<Integer> duplicates = new LinkedHashSet<>();
292 
293         if (arr == null)
294             return duplicates;
295 
296         for (int num : arr) {
297             if (!set.add(num)) {
298                 duplicates.add(num);
299             }
300         }
301 
302         return duplicates;
303     }
304 
305 
    // ================================================================
    ==
306     // Question: Frequency Of Elements
307     // Input : {1, 2, 2, 3, 3, 3}
308     // Output: {1=1, 2=2, 3=3}
309     // Time Complexity : O(n)
310     // Space Complexity: O(n)
311 
    // ================================================================
    ==
312     public static Map<Integer, Integer> frequencyCount(int[] arr) {
313 
314         Map<Integer, Integer> map = new LinkedHashMap<>();
315 
316         if (arr == null)
317             return map;
318 
319         for (int num : arr) {
320             map.put(num, map.getOrDefault(num, 0) + 1);
321         }
322 
323         // First non-repeating element
324 //         for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
325 //             if (entry.getValue() == 1) {
326 //                 System.out.println("First Non-Repeating Element: " + entry.
    getKey());
327 //                 break;
328 //             }
329 //         }
330 
331 //         Max Count and Element
332 //         int maxCount = 0;
333 //         int maxElement = 0;
334 //
335 //         for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
336 //             if (entry.getValue() > maxCount) {
337 //                 maxCount = entry.getValue();
338 //                 maxElement = entry.getKey();
339 //             }
340 //         }
341 
342 //         MajorityElement in an Array n/2
```

```
343 //         if (maxCount > arr.length / 2) {
344 //             System.out.println("Majority Element: " + maxElement);
345 //         } else {
346 //             System.out.println("No Majority Element");
347 //         }
348 
349         return map;
350     }
351 
352 
    // ================================================================
    ==
353     // Question: Missing Numbers In Array
354     // Input : {1, 2, 4, 7, 10}
355     // Output: [3, 5, 6, 8, 9]
356     // Time Complexity : O(n)
357     // Space Complexity: O(n)
358 
    // ================================================================
    ==
359     public static List<Integer> findAllMissingNumbers(int[] arr) {
360 
361         List<Integer> result = new ArrayList<>();
362 
363         if (arr == null || arr.length == 0) {
364             return result;
365         }
366 
367         Set<Integer> set = new HashSet<>();
368 
369         int min = Integer.MAX_VALUE;
370         int max = Integer.MIN_VALUE;
371 
372         for (int num : arr) {
373             set.add(num);
374             min = Math.min(min, num);
375             max = Math.max(max, num);
376         }
377 
378         for (int i = min; i <= max; i++) {
379             if (!set.contains(i)) {
380                 result.add(i);
381             }
382         }
383 
384         return result;
385     }
386 
387 
    // ================================================================
    ==
388     // Question: Move Zeros To End
389     // Input : {0, 1, 0, 3, 12}
390     // Output: {1, 3, 12, 0, 0}
391     // Time Complexity : O(n)
392     // Space Complexity: O(1)
393 
```

```java

    // ================================================================================
    ==
    //    For left all condtion will be -- nonZeroPos = nums.length - 1 for loop
    starts nums.length-1, i>= 0, i--
    public static void moveZerosToEnd(int[] nums) {

        if (nums == null || nums.length <= 1)
            return;

        int nonZeroPos = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                int temp         = nums[i];
                nums[i]          = nums[nonZeroPos];
                nums[nonZeroPos] = temp;
                nonZeroPos++;
            }
        }
    }


    // ================================================================================
    ==
    // Question: Merge Two Arrays
    // Input : {1, 2, 3} and {4, 5, 6}
    // Output: {1, 2, 3, 4, 5, 6}
    // Time Complexity : O(n + m)
    // Space Complexity: O(n + m)

    // ================================================================================
    ==
    public static int[] mergeArrays(int[] arr1, int[] arr2) {

//        int[] result = new int[arr1.length + arr2.length];
//
//        for (int i = 0; i < arr1.length; i++) {
//            result[i] = arr1[i];
//        }
//
//        for (int i = 0; i < arr2.length; i++) {
//            result[arr1.length + i] = arr2[i];
//        }
//
//        return result;

        return IntStream.concat(
                Arrays.stream(arr1),
                Arrays.stream(arr2)
        ).toArray();
    }


    // ================================================================================
    ==
    // Question: Intersection Of Arrays
```

```
441     // Input : {1, 2, 3, 4, 5} and {4, 5, 6, 7}
442     // Output: {4, 5}
443     // Time Complexity : O(n + m)
444     // Space Complexity: O(n)
445 
    // ================================================================================
    ==
446     public static int[] intersection(int[] arr1, int[] arr2) {
447 
448         if (arr1 == null || arr2 == null) return new int[0];
449 
450         Set<Integer> set = Arrays.stream(arr1).boxed().collect(Collectors.
    toSet());
451 
452         return Arrays.stream(arr2)
453                 .distinct()
454                 .filter(set::contains)
455                 .toArray();
456     }
457 
458 
    // ================================================================================
    ==
459     // Question: Union Of Arrays
460     // Input : {1, 2, 3, 4, 5} and {4, 5, 6, 7}
461     // Output: {1, 2, 3, 4, 5, 6, 7}
462     // Time Complexity : O(n + m)
463     // Space Complexity: O(n + m)
464 
    // ================================================================================
    ==
465     public static int[] union(int[] arr1, int[] arr2) {
466 
467         if (arr1 == null) arr1 = new int[0];
468         if (arr2 == null) arr2 = new int[0];
469 
470         return IntStream.concat(
471                         Arrays.stream(arr1),
472                         Arrays.stream(arr2))
473                 .distinct()
474                 .toArray();
475     }
476 
477 
    // ================================================================================
    ==
478     // Question: Two Sum Problem
479     // Input : {2, 7, 11, 15}, target = 9
480     // Output: [0, 1]  (indices where arr[0]+arr[1] = 9)
481     // Time Complexity : O(n^2)
482     // Space Complexity: O(1)
483 
    // ================================================================================
    ==
484     public static List<int[]> allPairs(int[] nums, int target) {
485 
486         List<int[]> result = new ArrayList<>();
```

```java

        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    result.add(new int[]{i, j});
                }
            }
        }

        return result;
    }


    // ================================================================================
    ==
    // Question: Pair With Given Sum (Using HashSet)
    // Input : {1, 2, 3, 4, 5}, target = 7
    // Output: [2,5], [3,4]
    // Time Complexity : O(n)
    // Space Complexity: O(n)

    // ================================================================================
    ==
    public static List<int[]> findPairsWithSum(int[] arr, int target) {

        List<int[]> result = new ArrayList<>();
        Set<Integer> set = new HashSet<>();

        for (int num : arr) {

            int complement = target - num;

            if (set.contains(complement)) {
                result.add(new int[]{complement, num});
            }

            set.add(num);
        }

        return result;
    }


    // ================================================================================
    ==
    // Question: Top K Frequent Elements
    // Input : [1,1,1,2,2,3], k = 2
    // Output: [1,2]
    //
    // Explanation:
    // 1. Count the frequency using HashMap.
    // 2. Sort the map entries by frequency in descending order.
    // 3. Return the first k elements.
    //
    // Time Complexity : O(n log n)
    // Space Complexity: O(n)

```

```
537
    // ================================================================
    ==
538     public static List<Integer> topKFrequent(int[] nums, int k) {
539
540         Map<Integer, Integer> map = new HashMap<>();
541
542         // Count frequency
543         for (int num : nums) {
544             map.put(num, map.getOrDefault(num, 0) + 1);
545         }
546
547         List<Map.Entry<Integer, Integer>> list = new ArrayList<>(map.entrySet
    ());
548
549         // Sort by frequency (highest first)
550         list.sort((a, b) -> b.getValue() - a.getValue());
551
552         List<Integer> result = new ArrayList<>();
553
554         for (int i = 0; i < k; i++) {
555             result.add(list.get(i).getKey());
556         }
557
558         return result;
559     }
560
561
    // ================================================================
    ==
562     // Question: Maximum Subarray Sum  (Kadane's Algorithm)
563     // Input : {-2, 1, -3, 4, -1, 2, 1, -5, 4}
564     // Output: 6
565     // Subarray = {4, -1, 2, 1}
566     // Time Complexity : O(n)
567     // Space Complexity: O(1)
568
    // ================================================================
    ==
569     public static void maxSubArray(int[] arr) {
570
571         int currentSum = arr[0];
572         int maxSum = arr[0];
573
574         int start = 0;
575         int end = 0;
576         int tempStart = 0;
577         for (int i = 1; i < arr.length; i++) {
578
579             if (arr[i] > currentSum + arr[i]) {
580 //                Forget Prev array start with new array
581                 currentSum = arr[i];
582                 tempStart = i;
583             } else {
584 //                Prev array is imp continue
585                 currentSum += arr[i];
586             }
```

```
587
588             if (currentSum > maxSum) {
589                 maxSum = currentSum;
590                 start = tempStart;
591                 end = i;
592             }
593         }
594         System.out.println("Max Sum = " + maxSum);
595
596         for (int i = start; i <= end; i++) {
597             System.out.print(arr[i] + " ");
598         }
599     }
600
601
    // ================================================================
    ==
602     // Question: Subset Sum (Combination Sum)
603     // Input : nums = {2, 7, 3, 6, 4, 5}, target = 9
604     // Output: All combinations that sum to 9
605     //         e.g. [2, 7], [3, 6], [4, 5], [2, 3, 4] ...
606
    // ================================================================
    ==
607     public static class SubsetSum {
608
609         public static void findCombinations(int[] nums, int target) {
610             System.out.println("    Combinations that sum to " + target + ":"
    );
611             backtrack(nums, target, 0, new ArrayList<>());
612         }
613
614         private static void backtrack(int[] nums,
615                                       int target,
616                                       int start,
617                                       List<Integer> current) {
618
619             if (target == 0) {
620                 System.out.println("    " + current);
621                 return;
622             }
623
624             if (target < 0)
625                 return;
626
627             for (int i = start; i < nums.length; i++) {
628                 current.add(nums[i]);
629                 backtrack(nums, target - nums[i], i + 1, current);
630                 current.remove(current.size() - 1);
631             }
632         }
633     }
634
635
    // ================================================================
    ==
636     // Question: Container With Most Water
```

```
637     // Input : [1,8,6,2,5,4,8,3,7]
638     // Output: 49
639     //
640     // Explanation:
641     // Find two lines that together with the x-axis form a container
642     // that holds the maximum amount of water.
643     //
644     // Formula:
645     // Area = min(height[left], height[right]) * (right - left)
646     //
647     // Time Complexity : O(n)
648     // Space Complexity: O(1)
649
    // ================================================================
    ==
650     public static int maxArea(int[] height) {
651
652         int left = 0;
653         int right = height.length - 1;
654
655         int maxArea = 0;
656
657         while (left < right) {
658
659             int area = Math.min(height[left], height[right]) * (right - left
    );
660
661             maxArea = Math.max(maxArea, area);
662
663             if (height[left] < height[right]) {
664                 left++;
665             } else {
666                 right--;
667             }
668         }
669
670         return maxArea;
671     }
672
673
    // ================================================================
    ==
674     // Question: Remove Duplicates from Sorted Array
675     // Input : [1,1,2,2,3,4,4]
676     // Output: [1,2,3,4]
677     //
678     // Explanation:
679     // Remove duplicates in-place from a sorted array and return
680     // the number of unique elements.
681     //
682     // Time Complexity : O(n)
683     // Space Complexity: O(1)
684
    // ================================================================
    ==
685     public static int removeDuplicateTwoPointers(int[] nums) {
686
```

```
687         if (nums.length == 0) {
688             return 0;
689         }
690 
691         int i = 0;
692 
693         for (int j = 1; j < nums.length; j++) {
694 
695             if (nums[i] != nums[j]) {
696                 i++;
697                 nums[i] = nums[j];
698             }
699         }
700 
701         return i + 1;
702     }
703 
704 
    // ================================================================================
    ==
705     // Question: Search Insert Position
706     // Input : nums = [1,3,5,6], target = 5
707     // Output: 2
708     //
709     // Explanation:
710     // Return the index if the target is found.
711     // Otherwise, return the index where it should be inserted.
712     //
713     // Time Complexity : O(log n)
714     // Space Complexity: O(1)
715 
    // ================================================================================
    ==
716     public static int searchInsert(int[] nums, int target) {
717 
718         int left = 0;
719         int right = nums.length - 1;
720 
721         while (left <= right) {
722 
723             int mid = left + (right - left) / 2;
724 
725             if (nums[mid] == target) {
726                 return mid;
727             } else if (nums[mid] < target) {
728                 left = mid + 1;
729             } else {
730                 right = mid - 1;
731             }
732         }
733 
734         return left;
735     }
736 
737 
    // ================================================================================
    ==
```

```
738     // Question: First Occurrence of an Element
739     // Input : nums = [1,2,2,2,3,4], target = 2
740     // Output: 1
741     //
742     // Explanation:
743     // Find the first occurrence of the target in a sorted array.
744     //
745     // Time Complexity : O(log n)
746     // Space Complexity: O(1)
747 
    // ================================================================================
    ==
748     public static int firstOccurrence(int[] nums, int target) {
749 
750         int left = 0;
751         int right = nums.length - 1;
752         int result = -1;
753 
754         while (left <= right) {
755 
756             int mid = left + (right - left) / 2;
757 
758             if (nums[mid] == target) {
759                 result = mid;
760                 right = mid - 1;      // Search on left side
761             } else if (nums[mid] < target) {
762                 left = mid + 1;
763             } else {
764                 right = mid - 1;
765             }
766         }
767 
768         return result;
769     }
770 
771 
    // ================================================================================
    ==
772     // Question: Last Occurrence of an Element
773     // Input : nums = [1,2,2,2,3,4], target = 2
774     // Output: 3
775     //
776     // Explanation:
777     // Find the last occurrence of the target in a sorted array.
778     //
779     // Time Complexity : O(log n)
780     // Space Complexity: O(1)
781 
    // ================================================================================
    ==
782     public static int lastOccurrence(int[] nums, int target) {
783 
784         int left = 0;
785         int right = nums.length - 1;
786         int result = -1;
787 
788         while (left <= right) {
```

```
789
790             int mid = left + (right - left) / 2;
791
792             if (nums[mid] == target) {
793                 result = mid;
794                 left = mid + 1;      // Search on right side
795             } else if (nums[mid] < target) {
796                 left = mid + 1;
797             } else {
798                 right = mid - 1;
799             }
800         }
801
802         return result;
803     }
804
805
    // ================================================================================
    ==
806     // Question: Find Peak Element
807     // Input : [1,2,3,1]
808     // Output: 2
809     //
810     // Explanation:
811     // nums[2] = 3 is greater than its neighbors (2 and 1),
812     // so index 2 is the peak element.
813     //
814     // Time Complexity : O(log n)
815     // Space Complexity: O(1)
816
    // ================================================================================
    ==
817     public static int findPeakElement(int[] nums) {
818
819         int left = 0;
820         int right = nums.length - 1;
821
822         while (left < right) {
823
824             int mid = left + (right - left) / 2;
825
826             if (nums[mid] < nums[mid + 1]) {
827                 left = mid + 1;
828             } else {
829                 right = mid;
830             }
831         }
832
833         return left;
834     }
835
836
    // ================================================================================
    ==
837     // Question: Square Root (Integer)
838     // Input : 8
839     // Output: 2
```

```
840     //
841     // Explanation:
842     // Return the integer part (floor) of the square root.
843     //
844     // sqrt(8) = 2.828...
845     // Answer = 2
846     //
847     // Time Complexity : O(log n)
848     // Space Complexity: O(1)
849
    // ================================================================
    ==
850     public static int mySqrt(int x) {
851
852         if (x == 0 || x == 1) {
853             return x;
854         }
855
856         int left = 1;
857         int right = x;
858         int ans = 0;
859
860         while (left <= right) {
861
862             int mid = left + (right - left) / 2;
863
864             long square = (long) mid * mid;
865
866             if (square == x) {
867                 return mid;
868             } else if (square < x) {
869                 ans = mid;
870                 left = mid + 1;
871             } else {
872                 right = mid - 1;
873             }
874         }
875
876         return ans;
877     }
878
879
    // ================================================================
    ==
880     // Question: Next Greater Element (Brute Force)
881     // Input : [4, 5, 2, 10, 8]
882     // Output: [5, 10, 10, -1, -1]
883     //
884     // Explanation:
885     // For every element, check all elements on its right.
886     // The first greater element is the answer.
887     // If none is found, return -1.
888     //
889     // Time Complexity : O(n²)
890     // Space Complexity: O(1)
891
    // ================================================================
```

```
891 ==
892     public static int[] nextGreaterElement(int[] arr) {
893 
894         int[] result = new int[arr.length];
895 
896         for (int i = 0; i < arr.length; i++) {
897 
898             result[i] = -1;
899 
900             for (int j = i + 1; j < arr.length; j++) {
901 
902                 if (arr[j] > arr[i]) {
903                     result[i] = arr[j];
904                     break;
905                 }
906             }
907         }
908 
909         return result;
910     }
911 
912 
    // ============================================================================
    ==
913     // Question: Leaders In Array (Brute Force)
914     // Input : {16, 17, 4, 3, 5, 2}
915     // Output: [17, 5, 2]
916     //
917     // Explanation:
918     // A leader is an element that is greater than all the elements
919     // to its right.
920     //
921     // Time Complexity : O(n²)
922     // Space Complexity: O(1)
923 
    // ============================================================================
    ==
924     public static List<Integer> findLeaders(int[] arr) {
925 
926         List<Integer> leaders = new ArrayList<>();
927 
928         for (int i = 0; i < arr.length; i++) {
929 
930             boolean isLeader = true;
931 
932             for (int j = i + 1; j < arr.length; j++) {
933 
934                 if (arr[j] > arr[i]) {
935                     isLeader = false;
936                     break;
937                 }
938             }
939 
940             if (isLeader) {
941                 leaders.add(arr[i]);
942             }
943         }
```

```
944
945         return leaders;
946     }
947
948
    // ================================================================================
    ===
949     // Question: Sort 0s 1s 2s  (Dutch National Flag Algorithm)
950     // Input : {2, 0, 2, 1, 1, 0}
951     // Output: {0, 0, 1, 1, 2, 2}
952     // Time Complexity : O(n)
953     // Space Complexity: O(1)
954
    // ================================================================================
    ===
955     public static void sortZeroOneTwo(int[] arr) {
956
957         if (arr == null || arr.length <= 1) {
958             return;
959         }
960
961         int zero = 0;
962         int one = 0;
963         int two = 0;
964
965         // Count 0s, 1s, and 2s
966         for (int num : arr) {
967             if (num == 0) {
968                 zero++;
969             } else if (num == 1) {
970                 one++;
971             } else if (num == 2) {
972                 two++;
973             } else {
974                 throw new IllegalArgumentException(
975                         "Array should contain only 0, 1, and 2");
976             }
977         }
978
979         // Fill the array back
980         int index = 0;
981
982         for (int i = 0; i < zero; i++) {
983             arr[index++] = 0;
984         }
985
986         for (int i = 0; i < one; i++) {
987             arr[index++] = 1;
988         }
989
990         for (int i = 0; i < two; i++) {
991             arr[index++] = 2;
992         }
993
994         System.out.println(Arrays.toString(arr));
995     }
996
```

```
 997
    // ================================================================
    ===
 998     // Question: Product Of Array Except Self
 999     // Input : {1, 2, 3, 4}
1000     // Output: {24, 12, 8, 6}
1001     // (result[i] = product of all elements except arr[i])
1002     // Time Complexity : O(n^2)
1003     // Space Complexity: O(n)
1004
    // ================================================================
    ===
1005     public static int[] productExceptSelf(int[] nums) {
1006
1007         int totalProduct = 1;
1008
1009         for (int num : nums) {
1010             totalProduct *= num;
1011         }
1012
1013         int[] result = new int[nums.length];
1014
1015         for (int i = 0; i < nums.length; i++) {
1016             result[i] = totalProduct / nums[i];
1017         }
1018
1019         return result;
1020     }
1021
1022
    // ================================================================
    ===
1023     // Question: Best Time To Buy And Sell Stock
1024     // Input : {7, 1, 5, 3, 6, 4}
1025     // Output: 5
1026     // Buy  = 1  (buy at lowest price)
1027     // Sell = 6  (sell at highest price after buy)
1028     // Profit = Sell - Buy = 6 - 1 = 5
1029     // Time Complexity : O(n)
1030     // Space Complexity: O(1)
1031
    // ================================================================
    ===
1032     public static int maxProfit(int[] prices) {
1033
1034         if (prices == null || prices.length <= 1) return 0;
1035
1036         int minPrice  = prices[0];
1037         int buyDay    = 0;
1038         int sellDay   = 0;
1039         int maxProfit = 0;
1040
1041         for (int i = 1; i < prices.length; i++) {
1042
1043             if (prices[i] < minPrice) {
1044                 minPrice = prices[i];
1045                 buyDay   = i;
```

```
1046            }
1047
1048            if (prices[i] - minPrice > maxProfit) {
1049                maxProfit = prices[i] - minPrice;
1050                sellDay   = i;
1051            }
1052        }
1053
1054        System.out.println("    Buy  Day   : Day " + buyDay  + "  -> Price
     = " + prices[buyDay]);
1055        System.out.println("    Sell Day   : Day " + sellDay + "  -> Price
     = " + prices[sellDay]);
1056        System.out.println("    Max Profit : " + prices[sellDay] + " - " +
    prices[buyDay] + " = " + maxProfit);
1057
1058        return maxProfit;
1059    }
1060
1061
    // ================================================================
    ===
1062    // Question: First Non-Repeating Element (Already done above)
1063    // Input : {9, 4, 9, 6, 7, 4}
1064    // Output: 6
1065    // Time Complexity : O(n)
1066    // Space Complexity: O(n)
1067
    // ================================================================
    ===
1068    public static Integer firstNonRepeating(int[] arr) {
1069
1070        if (arr == null || arr.length == 0) return null;
1071
1072        Map<Integer, Integer> map = new LinkedHashMap<>();
1073
1074        for (int num : arr) {
1075            map.put(num, map.getOrDefault(num, 0) + 1);
1076        }
1077
1078        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
1079            if (entry.getValue() == 1) {
1080                return entry.getKey();
1081            }
1082        }
1083
1084        return null;
1085    }
1086
1087
    // ================================================================
    ===
1088    // Question: Contains Duplicate
1089    // Input : {1, 2, 3, 1}
1090    // Output: true
1091    // Time Complexity : O(n)
1092    // Space Complexity: O(n)
1093
```

```
1093
     // ================================================================
     ===
1094     public static boolean containsDuplicate(int[] arr) {
1095
1096         if (arr == null || arr.length <= 1) return false;
1097
1098         return Arrays.stream(arr).distinct().count() != arr.length;
1099     }
1100
1101
     // ================================================================
     ===
1102     // Question: Sort Array Ascending (Using Stream)
1103     // Input : {5, 3, 8, 1, 2}
1104     // Output: {1, 2, 3, 5, 8}
1105
     // ================================================================
     ===
1106     public static int[] sortAscendingStream(int[] arr) {
1107
1108         if (arr == null) return null;
1109
1110         return Arrays.stream(arr).sorted().toArray();
1111     }
1112
1113
     // ================================================================
     ===
1114     // Question: Sort Array Descending (Using Stream)
1115     // Input : {5, 3, 8, 1, 2}
1116     // Output: {8, 5, 3, 2, 1}
1117
     // ================================================================
     ===
1118     public static int[] sortDescendingStream(int[] arr) {
1119
1120         if (arr == null) return null;
1121
1122 //         Integer[] arr = {5, 3, 8, 1, 2};
1123 //
1124 //         Arrays.sort(arr, Collections.reverseOrder());
1125 //
1126 //         System.out.println(Arrays.toString(arr));
1127
1128         return Arrays.stream(arr)
1129                 .boxed()
1130                 .sorted(Collections.reverseOrder())
1131                 .mapToInt(Integer::intValue)
1132                 .toArray();
1133     }
1134
1135
     // ================================================================
     ===
1136     // Question: Separate Odd and Even Numbers (Using Stream)
1137     // Input : {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
1138     // Output: Even = [2, 4, 6, 8, 10]  Odd = [1, 3, 5, 7, 9]
1139
     // ============================================================================
     ===
1140     public static void findOddEvenNumbers(int[] nums) {
1141
1142         if (nums == null) return;
1143
1144         List<Integer> evenNumbers = Arrays.stream(nums)
1145                 .boxed()
1146                 .filter(n -> n % 2 == 0)
1147                 .collect(Collectors.toList());
1148
1149         List<Integer> oddNumbers = Arrays.stream(nums)
1150                 .boxed()
1151                 .filter(n -> n % 2 != 0)
1152                 .collect(Collectors.toList());
1153
1154         System.out.println("    Even Numbers : " + evenNumbers);
1155         System.out.println("    Odd Numbers  : " + oddNumbers);
1156     }
1157
1158
     // ============================================================================
     ===
1159     // Question: Sort Map By Values
1160     // Input : {A=3, B=1, C=2, D=1}
1161     // Output: {B=1, D=1, C=2, A=3}
1162
     // ============================================================================
     ===
1163     public static void sortMapByValues(Map<String, Integer> map) {
1164
1165         if (map == null) return;
1166
1167         Map<String, Integer> sortedMap = map.entrySet()
1168                 .stream()
1169                 .sorted(Map.Entry.comparingByValue())
1170                 .collect(Collectors.toMap(
1171                         Map.Entry::getKey,
1172                         Map.Entry::getValue,
1173                         (oldValue, newValue) -> oldValue,
1174                         LinkedHashMap::new
1175                 ));
1176
1177         System.out.println("    Sorted Map : " + sortedMap);
1178     }
1179
1180
     // ============================================================================
     ===
1181     // Question: Sort Map By Value Then Key
1182     // Input : {C=2, A=1, D=2, B=1}
1183     // Output: {A=1, B=1, C=2, D=2}
1184
     // ============================================================================
     ===
```

```java
    public static void sortMapByValueThenKey(Map<String, Integer> map) {

        if (map == null) return;

        Map<String, Integer> sortedMap = map.entrySet()
                .stream()
                .sorted(
                        Map.Entry.<String, Integer>comparingByValue()
                                .thenComparing(Map.Entry.comparingByKey())
                )
                .collect(Collectors.toMap(
                        Map.Entry::getKey,
                        Map.Entry::getValue,
                        (oldValue, newValue) -> oldValue,
                        LinkedHashMap::new
                ));

        System.out.println("    Sorted Map : " + sortedMap);
    }


    // ================================================================================
    ===
    // Question: Rotate Array By K Positions (Right Rotation)
    // Input : {1, 2, 3, 4, 5, 6, 7}, k = 3
    // Output: {5, 6, 7, 1, 2, 3, 4}
    // Time Complexity : O(n)
    // Space Complexity: O(1)

    // ================================================================================
    ===
    public static void rotateArrayByK(int[] nums, int k) {

        if (nums == null || nums.length == 0 || k < 0)
            return;

        k = k % nums.length;
        reverseSubArray(nums, 0, nums.length - 1);
        reverseSubArray(nums, 0, k - 1);
        reverseSubArray(nums, k, nums.length - 1);
    }


    // ================================================================================
    ===
    // Question: Rotate Array By K Positions (Left Rotation)
    // Input : {1, 2, 3, 4, 5, 6, 7}, k = 3
    // Output: {4, 5, 6, 7, 1, 2, 3}
    // Time Complexity : O(n)
    // Space Complexity: O(1)

    // ================================================================================
    ===
    public static void rotateArrayLeftByK(int[] nums, int k) {

        if (nums == null || nums.length == 0 || k < 0)
```

```
1234            return;
1235
1236        k = k % nums.length;
1237
1238        reverseSubArray(nums, 0, k - 1);
1239        reverseSubArray(nums, k, nums.length - 1);
1240        reverseSubArray(nums, 0, nums.length - 1);
1241    }
1242
1243    private static void reverseSubArray(int[] nums, int start, int end) {
1244        while (start < end) {
1245            int temp    = nums[start];
1246            nums[start] = nums[end];
1247            nums[end]   = temp;
1248            start++;
1249            end--;
1250        }
1251    }
1252
1253
    // ============================================================================
    ===
1254    // Question: Find Missing Single Number (1 to N sequence)
1255    // Input : {1, 2, 4, 5, 6}, n = 6
1256    // Output: 3
1257    // Time Complexity : O(n)
1258    // Space Complexity: O(1)
1259
    // ============================================================================
    ===
1260    public static int findSingleMissingNumber(int[] arr, int n) {
1261
1262        int expectedSum = n * (n + 1) / 2;
1263        int actualSum   = 0;
1264
1265        for (int num : arr) {
1266            actualSum += num;
1267        }
1268
1269        return expectedSum - actualSum;
1270    }
1271
1272
    // ============================================================================
    ===
1273    // Question: Find Maximum Consecutive Occurrence of 1
1274    // Input : "110111101"
1275    // Output: 4
1276    // Time Complexity : O(n)
1277    // Space Complexity: O(1)
1278
    // ============================================================================
    ===
1279    public static int findMaxConsecutiveOnes(String str) {
1280
1281        int maxCount = 0;
1282        int currentCount = 0;
```

```
1283
1284            for (char ch : str.toCharArray()) {
1285
1286                if (ch == '1') {
1287                    currentCount++;
1288                    maxCount = Math.max(maxCount, currentCount);
1289                } else {
1290                    currentCount = 0;
1291                }
1292            }
1293
1294            return maxCount;
1295        }
1296
1297
        // ================================================================
        ===
1298        // Question: Flatten Nested List
1299        // Input : [[1, 2], [3, 4, 5]]
1300        // Output: [1, 2, 3, 4, 5]
1301
        // ================================================================
        ===
1302        public static List<Integer> flattenList(List<List<Integer>> nestedList
        ) {
1303
1304            if (nestedList == null) return Collections.emptyList();
1305
1306            return nestedList.stream()
1307                    .flatMap(List::stream)
1308                    .collect(Collectors.toList());
1309        }
1310
1311
        // ================================================================
        ===
1312        // Question: Split Array into Buckets of Fixed Size
1313        // Input : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Bucket Size = 3
1314        // Output: [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10]]
1315
        // ================================================================
        ===
1316    //    List<int[]> buckets = splitArray(arr, bucketSize);
1317    //
1318    //    for (int[] bucket : buckets) {
1319    //        System.out.println(Arrays.toString(bucket));
1320    //    }
1321        public static List<int[]> splitArray(int[] arr, int bucketSize) {
1322            List<int[]> result = new ArrayList<>();
1323
1324            for (int i = 0; i < arr.length; i += bucketSize) {
1325                result.add(Arrays.copyOfRange(
1326                        arr,
1327                        i,
1328                        Math.min(i + bucketSize, arr.length)
1329                ));
1330            }
```

```
1331
1332            return result;
1333        }
1334
1335
        // ================================================================
        ===
1336        // Question: Check Whether a Palindrome Can Be Formed from a String
1337        // Input : "aabb"
1338        // Output: true
1339        //
1340        // Input : "aabbc"
1341        // Output: true
1342        //
1343        // Input : "abc"
1344        // Output: false
1345
        // ================================================================
        ===
1346        public static boolean canFormPalindrome(String str) {
1347
1348            Map<Character, Integer> map = new HashMap<>();
1349
1350            for (char ch : str.toCharArray()) {
1351                map.put(ch, map.getOrDefault(ch, 0) + 1);
1352            }
1353
1354            int oddCount = 0;
1355
1356            for (int count : map.values()) {
1357                if (count % 2 != 0) {
1358                    oddCount++;
1359                }
1360            }
1361
1362            return oddCount <= 1;
1363        }
1364
1365
        // ================================================================
        ===
1366        // Question: Minimum Size Subarray Sum
1367        // Input : [2,3,1,2,4,3], Target = 7
1368        // Output: 2
1369        // Explanation: The subarray [4,3] has the minimum length (2)
1370
        // ================================================================
        ===
1371        public static int minSubArrayLen(int target, int[] nums) {
1372
1373            int left = 0;
1374            int sum = 0;
1375            int minLength = Integer.MAX_VALUE;
1376
1377            for (int right = 0; right < nums.length; right++) {
1378
1379                sum += nums[right];
```

```
1380
1381            while (sum >= target) {
1382
1383                minLength = Math.min(minLength, right - left + 1);
1384
1385                sum -= nums[left];
1386                left++;
1387            }
1388        }
1389
1390        return minLength == Integer.MAX_VALUE ? 0 : minLength;
1391    }
1392
1393
    // ================================================================
    ===
1394    // Question: Find Kth Largest Element
1395    // Input : [3,2,1,5,6,4], k = 2
1396    // Output: 5
1397
    // ================================================================
    ===
1398    public static int findKthLargest(int[] nums, int k) {
1399
1400        Arrays.sort(nums);
1401
1402        return nums[nums.length - k];
1403    }
1404
1405
    // ================================================================
    ===
1406    // Question: Factorial
1407    // Input : 5
1408    // Output: 120
1409
    // ================================================================
    ===
1410    public static long factorial(int n) {
1411
1412        if (n < 0)
1413            throw new IllegalArgumentException("Negative numbers not allowed
    ");
1414
1415        long factorial = 1;
1416
1417        for (int i = 1; i <= n; i++) {
1418            factorial *= i;
1419        }
1420
1421        return factorial;
1422    }
1423
1424
    // ================================================================
    ===
1425    // Question: Prime Number
```

```
1426     // Input : 29
1427     // Output: true
1428 
     // ============================================================================
     ===
1429     public static boolean isPrime(int n) {
1430 
1431         if (n <= 1)
1432             return false;
1433 
1434         for (int i = 2; i <= Math.sqrt(n); i++) {
1435             if (n % i == 0)
1436                 return false;
1437         }
1438 
1439         return true;
1440     }
1441 
1442 
     // ============================================================================
     ===
1443     // Question: Print Fibonacci Series
1444     // Input : 10
1445     // Output: 0 1 1 2 3 5 8 13 21 34
1446 
     // ============================================================================
     ===
1447     public static void printFibonacciSeries(int n) {
1448 
1449         int first = 0;
1450         int second = 1;
1451 
1452         for (int i = 0; i < n; i++) {
1453 
1454             System.out.print(first + " ");
1455 
1456             int next = first + second;
1457             first = second;
1458             second = next;
1459         }
1460     }
1461 
1462 
     // ============================================================================
     ===
1463     // Question: Power (Using Recursion)
1464     // Input : x = 2, n = 5
1465     // Output: 32
1466     //
1467     // Explanation:
1468     // Multiply x recursively until the exponent becomes 0.
1469     //
1470     // Time Complexity : O(n)
1471     // Space Complexity: O(n) (Recursive Call Stack)
1472 
     // ============================================================================
     ===
```

```
1473      public static long power(int x, int n) {
1474
1475          if (n == 0) {
1476              return 1;
1477          }
1478
1479          return x * power(x, n - 1);
1480      }
1481
1482      public static void printEvenFibonacciNumbers(int num) {
1483          int first = 0;
1484          int second = 1;
1485          int evenCount = 0;
1486
1487          while (evenCount < num) {
1488              if (first % 2 == 0) {
1489                  System.out.print(first + " ");
1490                  evenCount++;
1491              }
1492
1493              int next = first + second;
1494              first = second;
1495              second = next;
1496          }
1497      }
1498
1499
1500
      // ================================================================
      ===
1501      // Question: Print Even and Odd Numbers Using Two Threads
1502
      // ================================================================
      ===
1503      public static void printEvenOddUsingThreads(int max) {
1504
1505          Object lock     = new Object();
1506          int[]  counter  = {1};
1507
1508          Thread oddThread = new Thread(() -> {
1509              synchronized (lock) {
1510                  while (counter[0] <= max) {
1511                      if (counter[0] % 2 == 0) {
1512                          try { lock.wait(); } catch (InterruptedException e
      ) { Thread.currentThread().interrupt(); }
1513                      } else {
1514                          System.out.println("    Odd  Thread : " + counter[0
      ]);
1515                          counter[0]++;
1516                          lock.notify();
1517                      }
1518                  }
1519              }
1520          });
1521
1522          Thread evenThread = new Thread(() -> {
1523              synchronized (lock) {
```

```
1524                 while (counter[0] <= max) {
1525                     if (counter[0] % 2 != 0) {
1526                         try { lock.wait(); } catch (InterruptedException e
     ) { Thread.currentThread().interrupt(); }
1527                     } else {
1528                         System.out.println("    Even Thread : " + counter[0
     ]);
1529                         counter[0]++;
1530                         lock.notify();
1531                     }
1532                 }
1533             }
1534         });
1535 
1536         oddThread.start();
1537         evenThread.start();
1538     }
1539 
1540 
     // ============================================================================
     ===
1541     // Employee Class — used for Stream-based questions below
1542 
     // ============================================================================
     ===
1543     public static class Employee {
1544 
1545         private String name;
1546         private String department;
1547         private double salary;
1548 
1549         public Employee(String name, String department, double salary) {
1550             this.name       = name;
1551             this.department = department;
1552             this.salary     = salary;
1553         }
1554 
1555         public String getName()       { return name; }
1556         public String getDepartment() { return department; }
1557         public double getSalary()     { return salary; }
1558 
1559         @Override
1560         public String toString() {
1561             return name + " (" + salary + ")";
1562         }
1563     }
1564 
1565 //            | Requirement     | Collector
                                                                  |
1566 //            | --------------- | ------------------------------------------
     ------------------- |
1567 //            | Count employees | `Collectors.counting
     ()`                                          |
1568 //            | Average salary  | `Collectors.averagingDouble(Employee::
     getSalary)`                |
1569 //            | Total salary    | `Collectors.summingDouble(Employee::
     getSalary)`                  |
```

```
1570 //              | Highest salary  | `Collectors.maxBy(Comparator.comparing(
     Employee::getSalary))` |
1571 //              | Lowest salary   | `Collectors.minBy(Comparator.comparing(
     Employee::getSalary))` |
1572
1573
1574
     // ================================================================
     ===
1575     // Question: Group Employees By Department (Stream)
1576
     // ================================================================
     ===
1577     public static Map<String, List<Employee>> groupEmployeesByDept(List<
     Employee> employees) {
1578
1579         if (employees == null) return Collections.emptyMap();
1580
1581         return employees.stream()
1582                 .collect(Collectors.groupingBy(Employee::getDepartment));
1583     }
1584
1585
     // ================================================================
     ===
1586     // Question: Highest Salary Employee By Department (Stream)
1587
     // ================================================================
     ===
1588     public static Map<String, Optional<Employee>> highestSalaryByDept(List<
     Employee> employees) {
1589
1590         if (employees == null) return Collections.emptyMap();
1591
1592         return employees.stream()
1593                 .collect(Collectors.groupingBy(
1594                         Employee::getDepartment,
1595                         Collectors.maxBy(Comparator.comparing(Employee::
     getSalary))
1596                 ));
1597     }
1598
1599
     // ================================================================
     ===
1600     // Question: Employee With Highest Salary
1601
     // ================================================================
     ===
1602     public static Optional<Employee> highestSalary(List<Employee> employees
     ) {
1603
1604         return employees.stream()
1605                 .max(Comparator.comparing(Employee::getSalary));
1606     }
1607
1608
```

```
1608
1609    // Question: Second Highest Salary Employee
1610
1611    public static Optional<Employee> secondHighestSalary(List<Employee> employees) {
1612
1613        return employees.stream()
1614                .sorted(Comparator.comparing(Employee::getSalary).reversed())
1615                .skip(1)
1616                .findFirst();
1617    }
1618
1619
1620    // Question: Top 3 Highest Paid Employees
1621
1622    public static List<Employee> top3Employees(List<Employee> employees) {
1623
1624        return employees.stream()
1625                .sorted(Comparator.comparing(Employee::getSalary).reversed())
1626                .limit(3)
1627                .collect(Collectors.toList());
1628    }
1629
1630
1631    // Question: Count Employees In Each Department
1632
1633    public static Map<String, Long> countByDepartment(List<Employee> employees) {
1634
1635        return employees.stream()
1636                .collect(Collectors.groupingBy(
1637                        Employee::getDepartment,
1638                        Collectors.counting()));
1639    }
1640
1641
1642    // Question: Average Salary By Department
1643
1644    public static Map<String, Double> averageSalaryByDept(List<Employee>
```

```
1644 employees) {
1645 
1646         return employees.stream()
1647                 .collect(Collectors.groupingBy(
1648                         Employee::getDepartment,
1649                         Collectors.averagingDouble(Employee::getSalary)));
1650     }
1651 
1652 
     // ========================================================================
     ===
1653     // Question: Total Salary By Department
1654 
     // ========================================================================
     ===
1655     public static Map<String, Double> totalSalaryByDept(List<Employee>
     employees) {
1656 
1657         return employees.stream()
1658                 .collect(Collectors.groupingBy(
1659                         Employee::getDepartment,
1660                         Collectors.summingDouble(Employee::getSalary)));
1661     }
1662 
1663 
     // ========================================================================
     ===
1664     // Question: Employees Earning More Than 50000
1665 
     // ========================================================================
     ===
1666     public static List<Employee> highEarners(List<Employee> employees) {
1667 
1668         return employees.stream()
1669                 .filter(e -> e.getSalary() > 50000)
1670                 .collect(Collectors.toList());
1671     }
1672 
1673 
     // ========================================================================
     ===
1674     // Question: Sort Employees By Salary Descending
1675 
     // ========================================================================
     ===
1676     public static List<Employee> sortBySalaryDesc(List<Employee> employees
     ) {
1677 
1678         return employees.stream()
1679                 .sorted(Comparator.comparing(Employee::getSalary).reversed
     ())
1680                 .collect(Collectors.toList());
1681     }
1682 
1683 
     // ========================================================================
     ===
```

```
1684     // Question: Sort By Department Then Salary
1685 
     // ================================================================
     ===
1686     public static List<Employee> sortDeptSalary(List<Employee> employees) {
1687 
1688         return employees.stream()
1689                 .sorted(Comparator.comparing(Employee::getDepartment)
1690                         .thenComparing(Employee::getSalary))
1691                 .collect(Collectors.toList());
1692     }
1693 
1694 
     // ================================================================
     ===
1695     // Question: Get Distinct Departments
1696 
     // ================================================================
     ===
1697     public static List<String> departments(List<Employee> employees) {
1698 
1699         return employees.stream()
1700                 .map(Employee::getDepartment)
1701                 .distinct()
1702                 .collect(Collectors.toList());
1703     }
1704 
1705 
     // ================================================================
     ===
1706     // Question: Find Employees In IT Department
1707 
     // ================================================================
     ===
1708     public static List<Employee> itEmployees(List<Employee> employees) {
1709 
1710         return employees.stream()
1711                 .filter(e -> "IT".equals(e.getDepartment()))
1712                 .collect(Collectors.toList());
1713     }
1714 
1715 
     // ================================================================
     ===
1716     // Question: Group Employee Names By Department
1717 
     // ================================================================
     ===
1718     public static Map<String, List<String>> namesByDept(List<Employee>
     employees) {
1719 
1720         return employees.stream()
1721                 .collect(Collectors.groupingBy(
1722                         Employee::getDepartment,
1723                         Collectors.mapping(Employee::getName, Collectors.
     toList())));
1724     }
```

```
1725
1726
    // ================================================================
    ===
1727    // Question: Highest Salary Employee Department-Wise
1728
    // ================================================================
    ===
1729    public static Map<String, Optional<Employee>>
    highestSalaryDepartmentWise(List<Employee> employees) {
1730
1731        return employees.stream()
1732                .collect(Collectors.groupingBy(
1733                        Employee::getDepartment,
1734                        Collectors.maxBy(
1735                                Comparator.comparing(Employee::getSalary)
1736                        )
1737                ));
1738    }
1739
1740
    // ================================================================
    ===
1741    // Question: Employees Whose Name Starts With "A"
1742
    // ================================================================
    ===
1743    public static List<Employee> nameStartsWithA(List<Employee> employees) {
1744
1745        return employees.stream()
1746                .filter(e -> e.getName().startsWith("A"))
1747                .collect(Collectors.toList());
1748    }
1749
1750
    // ================================================================
    ===
1751    // Question: Highest Paid Employee Name Only
1752
    // ================================================================
    ===
1753    public static String highestPaidEmployeeName(List<Employee> employees) {
1754
1755        return employees.stream()
1756                .max(Comparator.comparing(Employee::getSalary))
1757                .map(Employee::getName)
1758                .orElse("");
1759    }
1760
1761
    // ================================================================
    ===
1762    // Question: Partition Employees By Salary > 50000
1763
    // ================================================================
    ===
1764    public static Map<Boolean, List<Employee>> partitionEmployees(List<
```

```
Employee> employees) {

        return employees.stream()
                .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000));
    }


    // ================================================================================
    public static Set<String> duplicateNames(List<Employee> employees) {

        Set<String> seen = new HashSet<>();

        return employees.stream()
                .map(Employee::getName)
                .filter(name -> !seen.add(name))
                .collect(Collectors.toSet());
    }


    // ================================================================================
    // Question: Find Nth Highest Salary Employee
    // Input : employees list, n = 2
    // Output: Employee with 2nd highest salary

    // ================================================================================
    public static Optional<Employee> nthHighestSalary(List<Employee> employees, int n) {

        return employees.stream()
                .sorted(Comparator.comparing(Employee::getSalary).reversed())
                .skip(n - 1)
                .findFirst();
    }


    // ================================================================================
    // Question: LRU Cache  (Least Recently Used Cache)
    // Uses LinkedHashMap with access-order = true
    // Automatically evicts the least recently used entry when capacity is exceeded

    // ================================================================================
    public static class LRUCache<K, V> extends LinkedHashMap<K, V> {

        private final int capacity;

```

```
1805        public LRUCache(int capacity) {
1806            super(capacity, 0.75f, true); // true = access-order
1807            this.capacity = capacity;
1808        }
1809
1810        @Override
1811        protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
1812            return size() > capacity;
1813        }
1814    }
1815
1816
    // ================================================================
    ===
1817    //                              RUN ALL METHODS
1818
    // ================================================================
    ===
1819    public static void run() {
1820
1821        System.out.println();
1822        System.out.println(
    "============================================================");
1823        System.out.println("            ARRAY CODING QUESTIONS - ALL METHODS
                ");
1824        System.out.println(
    "============================================================");
1825
1826        int[] arr = {10, 20, 30, 40, 50};
1827
1828
    // ------------------------------------------------------------
1829        System.out.println("\n1.  Find Largest Element");
1830        System.out.println("    Input  : " + Arrays.toString(arr));
1831        System.out.println("    Output : " + findLargest(arr));
1832
1833
    // ------------------------------------------------------------
1834        System.out.println("\n2.  Find Smallest Element");
1835        System.out.println("    Input  : " + Arrays.toString(arr));
1836        System.out.println("    Output : " + findSmallest(arr));
1837
1838
    // ------------------------------------------------------------
1839        System.out.println("\n3.  Find Second Largest Element");
1840        System.out.println("    Input  : " + Arrays.toString(arr));
1841        System.out.println("    Output : " + findSecondLargest(arr));
1842
1843
    // ------------------------------------------------------------
1844        System.out.println("\n4.  Find Second Smallest Element");
1845        System.out.println("    Input  : " + Arrays.toString(arr));
1846        System.out.println("    Output : " + findSecondSmallest(arr));
1847
1848
    // ------------------------------------------------------------
1849        int[] reverseArr = {1, 2, 3, 4, 5};
```

```
1850        reverseArray(reverseArr);
1851        System.out.println("\n5.  Reverse Array");
1852        System.out.println("    Input  : [1, 2, 3, 4, 5]");
1853        System.out.println("    Output : " + Arrays.toString(reverseArr));
1854
1855
    // ------------------------------------------------------------
1856        System.out.println("\n6.  Sum Of Array Elements");
1857        System.out.println("    Input  : " + Arrays.toString(arr));
1858        System.out.println("    Output : " + sumOfElements(arr));
1859
1860
    // ------------------------------------------------------------
1861        System.out.println("\n7.  Average Of Array Elements");
1862        System.out.println("    Input  : " + Arrays.toString(arr));
1863        System.out.println("    Output : " + averageOfElements(arr));
1864
1865
    // ------------------------------------------------------------
1866        System.out.println("\n8.  Count Even And Odd Numbers");
1867        System.out.println("    Input  : [1, 2, 3, 4, 5, 6]");
1868        countEvenOdd(new int[]{1, 2, 3, 4, 5, 6});
1869
1870
    // ------------------------------------------------------------
1871        System.out.println("\n9.  Find Maximum And Minimum");
1872        System.out.println("    Input  : " + Arrays.toString(arr));
1873        int[] maxMin = findMaxAndMin(arr);
1874        System.out.println("    Max    : " + maxMin[0]);
1875        System.out.println("    Min    : " + maxMin[1]);
1876
1877
    // ------------------------------------------------------------
1878        System.out.println("\n10. Search Element In Array");
1879        System.out.println("    Input  : " + Arrays.toString(arr) + ",
    target = 30");
1880        System.out.println("    Output : " + searchElement(arr, 30));
1881
1882
    // ------------------------------------------------------------
1883        int[] dupArr = {1, 2, 2, 3, 4, 4, 5};
1884        System.out.println("\n11. Remove Duplicates From Array");
1885        System.out.println("    Input  : " + Arrays.toString(dupArr));
1886        System.out.println("    Output : " + Arrays.toString(
    removeDuplicates(dupArr)));
1887
1888
    // ------------------------------------------------------------
1889        System.out.println("\n12. Find Duplicate Elements");
1890        System.out.println("    Input  : " + Arrays.toString(dupArr));
1891        System.out.println("    Output : " + findDuplicates(dupArr));
1892
1893
    // ------------------------------------------------------------
1894        int[] freqArr = {1, 2, 2, 3, 3, 3};
1895        System.out.println("\n13. Frequency Of Elements");
1896        System.out.println("    Input  : " + Arrays.toString(freqArr));
```

```
1897        System.out.println("    Output : " + frequencyCount(freqArr));
1898
1899
    // ------------------------------------------------------------------
1900        int[] missingArr = {1, 2, 4, 7, 10};
1901        System.out.println("\n14. Find Missing Numbers In Array");
1902        System.out.println("    Input  : " + Arrays.toString(missingArr));
1903        System.out.println("    Output : " + findAllMissingNumbers(
    missingArr));
1904
1905
    // ------------------------------------------------------------------
1906        int[] zeros = {0, 1, 0, 3, 12};
1907        moveZerosToEnd(zeros);
1908        System.out.println("\n15. Move Zeros To End");
1909        System.out.println("    Input  : [0, 1, 0, 3, 12]");
1910        System.out.println("    Output : " + Arrays.toString(zeros));
1911
1912
    // ------------------------------------------------------------------
1913        System.out.println("\n18. Merge Two Arrays");
1914        System.out.println("    Input  : [1, 2, 3] and [4, 5, 6]");
1915        System.out.println("    Output : " + Arrays.toString(
1916                mergeArrays(new int[]{1, 2, 3}, new int[]{4, 5, 6})));
1917
1918
    // ------------------------------------------------------------------
1919        System.out.println("\n19. Intersection Of Arrays");
1920        System.out.println("    Input  : [1, 2, 3, 4, 5] and [4, 5, 6, 7]");
1921        System.out.println("    Output : " + Arrays.toString(
1922                intersection(new int[]{1, 2, 3, 4, 5}, new int[]{4, 5, 6, 7
    })));
1923
1924
    // ------------------------------------------------------------------
1925        System.out.println("\n20. Union Of Arrays");
1926        System.out.println("    Input  : [1, 2, 3, 4, 5] and [4, 5, 6, 7]");
1927        System.out.println("    Output : " + Arrays.toString(
1928                union(new int[]{1, 2, 3, 4, 5}, new int[]{4, 5, 6, 7})));
1929
1930
    // ------------------------------------------------------------------
1931        System.out.println("\n21. Two Sum Problem (Return Indices)");
1932        System.out.println("    Input  : [2, 7, 11, 15], target = 9");
1933        List<int[]> twoSumPairs = allPairs(new int[]{2, 7, 11, 15}, 9);
1934        List<String> twoSumStr = new ArrayList<>();
1935        for (int[] p : twoSumPairs) twoSumStr.add(Arrays.toString(p));
1936        System.out.println("    Output : " + twoSumStr + "  (indices)");
1937
1938
    // ------------------------------------------------------------------
1939        System.out.println("\n22. Pair With Given Sum (Return Values)");
1940        System.out.println("    Input  : [1, 2, 3, 4, 5], target = 7");
1941        List<int[]> pairValues = findPairsWithSum(new int[]{1, 2, 3, 4, 5},
    7);
1942        List<String> pairStr = new ArrayList<>();
1943        for (int[] p : pairValues) pairStr.add(Arrays.toString(p));
```

```
1944        System.out.println("    Output : " + pairStr);
1945
1946
    // ------------------------------------------------------------
1947        System.out.println("\n23. Maximum Subarray Sum (Kadane's Algorithm)"
    );
1948
1949        int[] arr1 = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
1950        System.out.println("    Input  : [-2, 1, -3, 4, -1, 2, 1, -5, 4]");
1951        maxSubArray(arr1);
1952        System.out.println("    (Best subarray = [4, -1, 2, 1])");
1953
1954
    // ------------------------------------------------------------
1955        System.out.println("\n24. Leaders In Array");
1956        System.out.println("    Input  : [16, 17, 4, 3, 5, 2]");
1957        System.out.println("    Output : " + findLeaders(
1958                new int[]{16, 17, 4, 3, 5, 2}));
1959
1960
1961
    // ------------------------------------------------------------
1962        int[] dnf = {2, 0, 2, 1, 1, 0};
1963        sortZeroOneTwo(dnf);
1964        System.out.println("\n26. Sort 0s 1s 2s (Dutch National Flag)");
1965        System.out.println("    Input  : [2, 0, 2, 1, 1, 0]");
1966        System.out.println("    Output : " + Arrays.toString(dnf));
1967
1968
    // ------------------------------------------------------------
1969        System.out.println("\n27. Product Of Array Except Self");
1970        System.out.println("    Input  : [1, 2, 3, 4]");
1971        System.out.println("    Output : " + Arrays.toString(
1972                productExceptSelf(new int[]{1, 2, 3, 4})));
1973
1974
    // ------------------------------------------------------------
1975        System.out.println("\n28. Best Time To Buy And Sell Stock");
1976        System.out.println("    Input  : [7, 1, 5, 3, 6, 4]");
1977        maxProfit(new int[]{7, 1, 5, 3, 6, 4});
1978
1979
    // ------------------------------------------------------------
1980        System.out.println("\n29. First Non-Repeating Element");
1981        System.out.println("    Input  : [9, 4, 9, 6, 7, 4]");
1982        System.out.println("    Output : " + firstNonRepeating(
1983                new int[]{9, 4, 9, 6, 7, 4}));
1984
1985
    // ------------------------------------------------------------
1986        System.out.println("\n30. Contains Duplicate");
1987        System.out.println("    Input  : [1, 2, 3, 1]");
1988        System.out.println("    Output : " + containsDuplicate(
1989                new int[]{1, 2, 3, 1}));
1990
1991
    // ------------------------------------------------------------
```

```
1992        System.out.println("\n31. Sort Array Ascending (Stream)");
1993        System.out.println("    Input  : [5, 3, 8, 1, 2]");
1994        System.out.println("    Output : " + Arrays.toString(
1995                sortAscendingStream(new int[]{5, 3, 8, 1, 2})));
1996
1997
    // ------------------------------------------------------------
1998        System.out.println("\n32. Sort Array Descending (Stream)");
1999        System.out.println("    Input  : [5, 3, 8, 1, 2]");
2000        System.out.println("    Output : " + Arrays.toString(
2001                sortDescendingStream(new int[]{5, 3, 8, 1, 2})));
2002
2003
    // ------------------------------------------------------------
2004        System.out.println("\n33. Separate Odd and Even Numbers (Stream)");
2005        System.out.println("    Input  : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]");
2006        findOddEvenNumbers(new int[]{1, 2, 3, 4, 5, 6, 7, 8, 9, 10});
2007
2008
    // ------------------------------------------------------------
2009        System.out.println("\n34. Next Greater Element");
2010        System.out.println("    Input  : [4, 5, 2, 10, 8]");
2011        System.out.println("    Output : " + Arrays.toString(
2012                nextGreaterElement(new int[]{4, 5, 2, 10, 8})));
2013
2014
    // ------------------------------------------------------------
2015        int[] rotateArr = {1, 2, 3, 4, 5, 6, 7};
2016        rotateArrayByK(rotateArr, 3);
2017        System.out.println("\n35. Rotate Array By K = 3 (Right Rotation)");
2018        System.out.println("    Input  : [1, 2, 3, 4, 5, 6, 7]");
2019        System.out.println("    Output : " + Arrays.toString(rotateArr));
2020
2021
    // ------------------------------------------------------------
2022        System.out.println("\n36. Find Missing Single Number (1 to N)");
2023        System.out.println("    Input  : [1, 2, 4, 5, 6], n = 6");
2024        System.out.println("    Output : " + findSingleMissingNumber(
2025                new int[]{1, 2, 4, 5, 6}, 6));
2026
2027
    // ------------------------------------------------------------
2028        List<List<Integer>> nested = Arrays.asList(
2029                Arrays.asList(1, 2),
2030                Arrays.asList(3, 4, 5));
2031        System.out.println("\n37. Flatten Nested List");
2032        System.out.println("    Input  : [[1, 2], [3, 4, 5]]");
2033        System.out.println("    Output : " + flattenList(nested));
2034
    // ------------------------------------------------------------
2035        List<Employee> emps = Arrays.asList(
2036                new Employee("Alice",   "IT",      70000),
2037                new Employee("Bob",     "HR",      50000),
2038                new Employee("Charlie", "IT",      90000),
2039                new Employee("David",   "Finance", 60000),
2040                new Employee("Eve",     "HR",      75000)
2041        );
```

```
2042
2043        System.out.println("\n38. Group Employees By Department");
2044        System.out.println("    Output : " + groupEmployeesByDept(emps));
2045
2046        System.out.println("\n39. Highest Salary Employee By Department");
2047        System.out.println("    Output : " + highestSalaryByDept(emps));
2048
2049 //        System.out.println("\n40. Employee With Highest Salary");
2050 //        System.out.println("    Output : " + highestSalary(emps).orElse(
     null));
2051 //
2052 //        System.out.println("\n41. Second Highest Salary Employee");
2053 //        System.out.println("    Output : " + secondHighestSalary(emps).
     orElse(null));
2054
2055        System.out.println("\n42. Top 3 Highest Paid Employees");
2056        System.out.println("    Output : " + top3Employees(emps));
2057
2058        System.out.println("\n43. Count Employees In Each Department");
2059        System.out.println("    Output : " + countByDepartment(emps));
2060
2061        System.out.println("\n44. Average Salary By Department");
2062        System.out.println("    Output : " + averageSalaryByDept(emps));
2063
2064        System.out.println("\n45. Total Salary By Department");
2065        System.out.println("    Output : " + totalSalaryByDept(emps));
2066
2067        System.out.println("\n46. Employees Earning More Than 50000");
2068        System.out.println("    Output : " + highEarners(emps));
2069
2070        System.out.println("\n47. Sort Employees By Salary Descending");
2071        System.out.println("    Output : " + sortBySalaryDesc(emps));
2072
2073        System.out.println("\n48. Distinct Departments");
2074        System.out.println("    Output : " + departments(emps));
2075
2076        System.out.println("\n49. Employees In IT Department");
2077        System.out.println("    Output : " + itEmployees(emps));
2078
2079        System.out.println("\n50. Nth Highest Salary (n=2)");
2080        System.out.println("    Output : " + nthHighestSalary(emps, 2).
     orElse(null));
2081
2082        System.out.println("\n53. Subset Sum Combinations");
2083        System.out.println("    Input  : nums = [2, 7, 3, 6, 4, 5], target
      = 9");
2084        SubsetSum.findCombinations(new int[]{2, 7, 3, 6, 4, 5}, 9);
2085
2086        System.out.println("\n
     ============================================================");
2087        System.out.println("                        ALL METHODS EXECUTED
      !                    ");
2088        System.out.println(
     "============================================================");
2089        System.out.println();
2090    }
2091
```

```
2092 }
2093
```